1. iroh-blob 프로젝트 개요 및 아키텍처

1.1 프로젝트 개요

iroh-blob이란?

iroh-blob은 **BLAKE3 해시 기반의 콘텐츠 주소 지정 가능한 데이터 전송 프로토콜**입니다. 이 시스템은 데이터의 내용을 기반으로 고유한 식별자(해시)를 생성하고, 이를 통해 데이터를 안전하고 효율적으로 전송할 수 있게 해줍니다.

핵심 특징

1. **콘텐츠 주소 지정 (Content Addressing)**

- 데이터의 BLAKE3 해시를 식별자로 사용
- 동일한 내용은 항상 같은 해시를 가짐
- 데이터 중복 제거와 무결성 보장

2. **검증된 스트리밍 (Verified Streaming)**

- 전송 중 실시간 데이터 무결성 검증
- 최대 16KiB 지연으로 오류 감지
- 부분 데이터에 대한 독립적 검증 가능

3. **효율적인 범위 요청**

- 파일의 특정 부분만 요청 가능
- 1024바이트 청크 단위의 세밀한 제어
- 불연속적인 범위 요청 지원

4. **확장 가능한 아키텍처**

- 테라바이트급 대용량 파일 지원
- 수백만 개의 작은 파일 효율적 처리
- 메모리 사용량 최적화

1.2 설계 철학

데이터 무결성 우선

iroh-blob은 **성능보다 데이터 무결성을 우선시**합니다. 모든 데이터는 전송 과정에서 검증되며, 잘못된 데이터가 감지되면 즉시 전송을 중단합니다.

```
// 데이터 무결성 검증 예시
if  computed_hash != expected_hash {
return Err(DecodeError::LeafHashMismatch { num: chunk_num });
}
```

크기 제한 없음

시스템은 블롭이나 컬렉션의 크기에 제한을 두지 않습니다. 테라바이트급 파일도 메모리에 전체를 로드하지 않고 스트리밍 방식으로 처리할 수 있습니다.

BLAKE3 특화

다른 해시 함수에 대한 일반화를 시도하지 않고, BLAKE3의 특별한 속성들을 최대한 활용합니다:

- 트리 구조 해싱
- 병렬 처리 가능
- 검증된 스트리밍 지원

1.3 전체 시스템 아키텍처

계층 구조

```
┌─────────────────────────────────────────────────────────────┐
│ Application Layer │
│ ┌──────────────────┐ ┌──────────────────────────────────┐ │
│ │ Client API │ │ Server API │ │
│ └──────────────────┘ └──────────────────────────────────┘ │
└─────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────┐
│ Protocol Layer │
│ ┌──────────────────┐ ┌──────────────────────────────────┐ │
│ │ Get Module │ │ Provider Module │ │
│ │ (Client Side) │ │ (Server Side) │ │
│ └──────────────────┘ └──────────────────────────────────┘ │
└─────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────┐
│ Network Layer │
│ QUIC Connections & Streams │
└─────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────┐
│ Storage Layer │
```

```
│ ┌──────────────┐ ┌──────────────┐ ┌──────────────────────────┐ │
│ │ Memory Store│ │ File Store │ │ Hybrid Store │ │
│ └──────────────┘ └──────────────┘ └──────────────────────────┘ │
└──────────────────────────────────────────────────────────────────┘
```

주요 구성 요소

1. Protocol Layer (프로토콜 계층)

- **역할**: 클라이언트-서버 간 통신 프로토콜 정의
- **주요 구성요소**:
- `GetRequest` : 데이터 요청 메시지
- `GetManyRequest` : 다중 블롭 요청
- `PushRequest` : 데이터 업로드 요청
- `ObserveRequest` : 실시간 상태 관찰

2. Provider Module (제공자 모듈)

- **역할**: 서버 측 요청 처리 및 데이터 제공
- **주요 기능**:
- 클라이언트 연결 관리
- 요청 파싱 및 검증
- 데이터 스트리밍 전송
- 진행 상황 추적

3. Get Module (요청 모듈)

- **역할**: 클라이언트 측 데이터 요청 및 수신

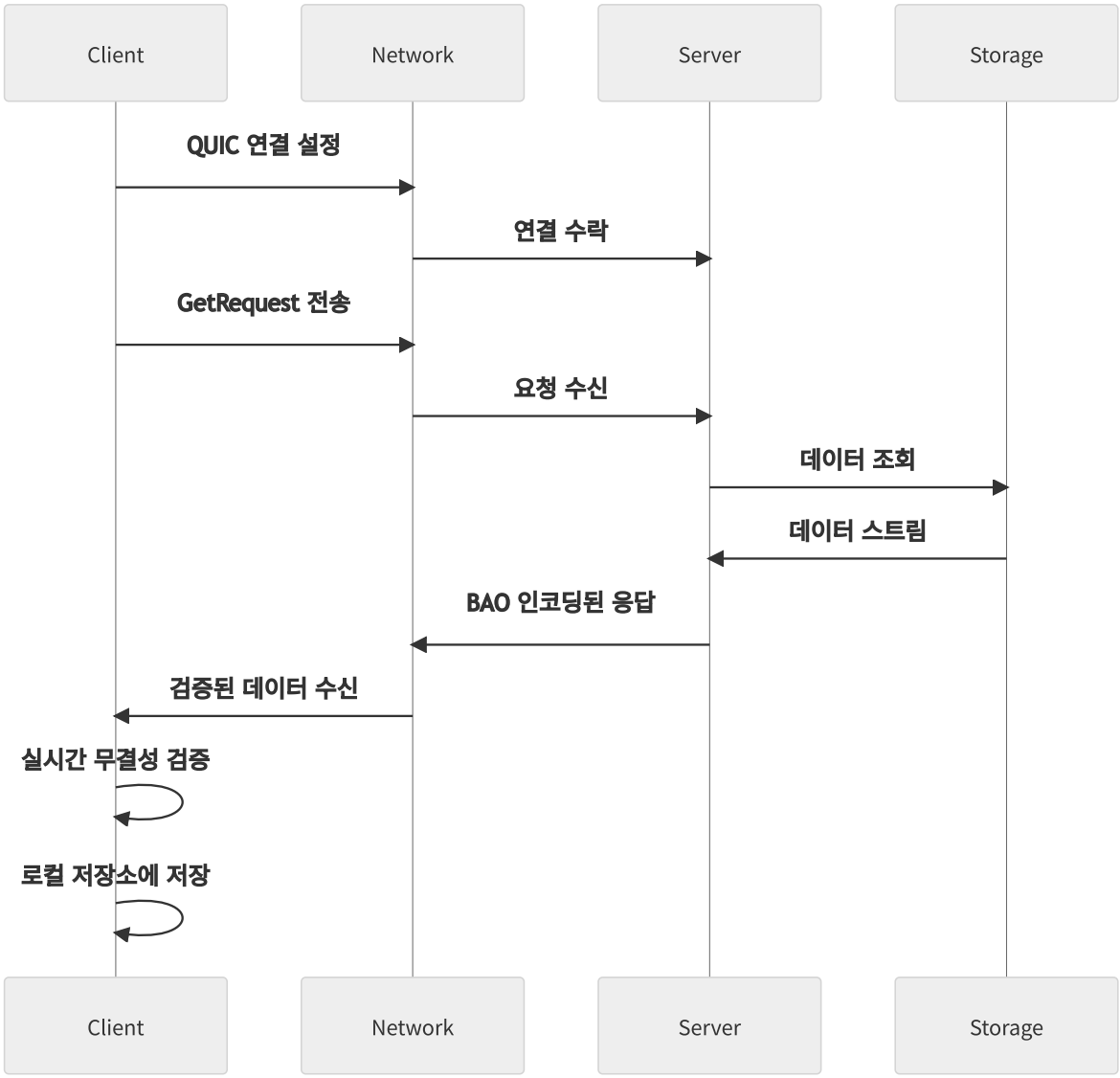
- **주요 기능**:
- 유한 상태 머신 기반 요청 처리
- 스트리밍 데이터 수신
- 실시간 무결성 검증
- 오류 처리 및 복구

4. Store Module (저장소 모듈)

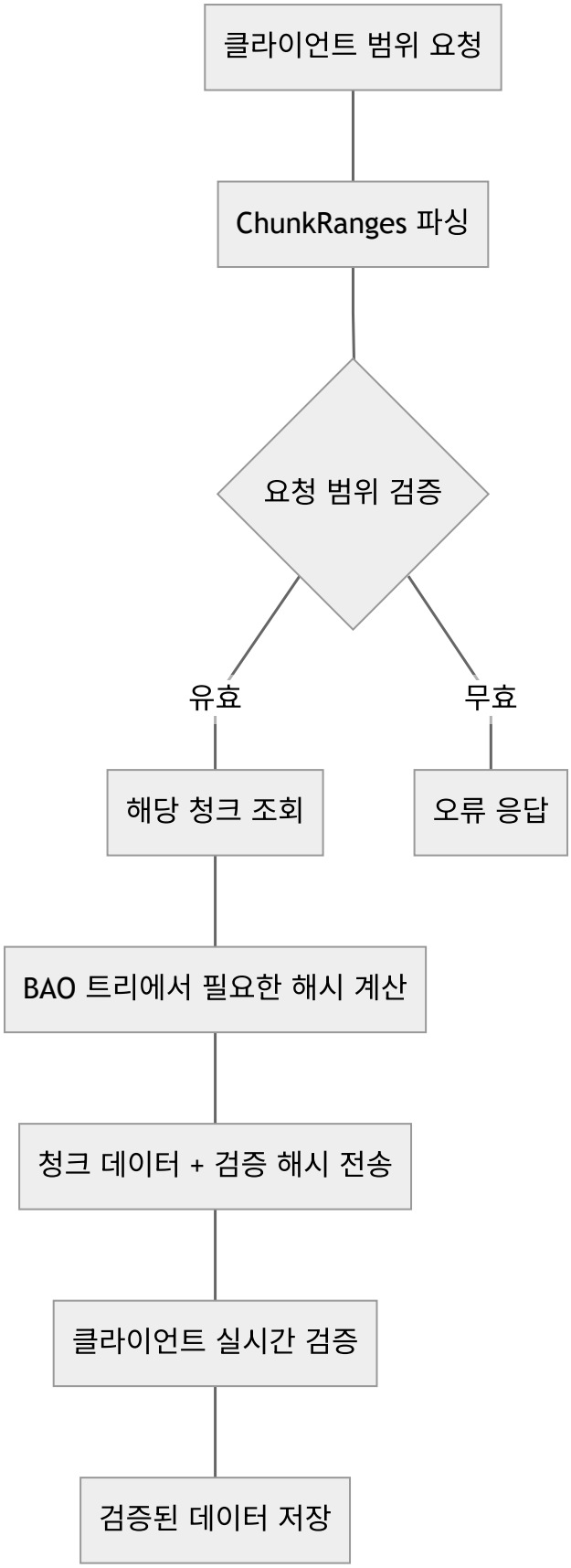
- **역할**: 다양한 저장소 구현체 제공
- **구현체**:
- `MemStore` : 메모리 기반 저장소
- `FsStore` : 파일 시스템 기반 저장소
- 하이브리드 접근법 지원

1.4 데이터 흐름 다이어그램

기본 Get 요청 흐름


Client
Network
Server
Storage
QUIC 연결 설정
연결 수락
GetRequest 전송
요청 수신
데이터 조회
데이터 스트림
BAO 인코딩된 응답
검증된 데이터 수신
실시간 무결성 검증
로컬 저장소에 저장
Client
Network
Server
Storage

범위 요청 처리 흐름



1.5 핵심 개념

BLAKE3 해시 트리

iroh-blob은 BLAKE3의 트리 구조를 활용하여 효율적인 부분 검증을 구현합니다:

```
Root Hash

/ \

Parent Hash Parent Hash

/ \ / \

Chunk 0 Chunk 1 Chunk 2 Chunk 3

(1024B) (1024B) (1024B) (1024B)
```

청크 그룹 최적화

16개의 연속된 청크(16KiB)를 하나의 그룹으로 처리하여 메타데이터 오버헤드를 줄입니다:

```
pub const IROH_BLOCK_SIZE: BlockSize = BlockSize::from_chunk_log(4); // 2^4 = 1
```

콘텐츠 주소 지정

데이터의 내용이 곧 주소가 되는 시스템:

```
let hash = blake3::hash(data); // 데이터 → 해시

let address = Hash::from(hash); // 해시 → 주소
```

1.6 사용 사례

1. 대용량 파일 배포

- 소프트웨어 업데이트
- 미디어 콘텐츠 배포
- 데이터셋 공유

2. 분산 저장 시스템

- 중복 제거 저장소
- 백업 시스템
- 콘텐츠 전송 네트워크

3. 버전 관리 시스템

- Git과 유사한 콘텐츠 추적
- 증분 업데이트
- 히스토리 관리

4. 실시간 스트리밍

- 라이브 데이터 피드
- 로그 스트리밍
- 센서 데이터 전송

이 개요를 통해 iroh-blob의 전체적인 구조와 설계 철학을 이해할 수 있습니다. 다음 장에서는 각 구성 요소의 세부 구현을 자세히 살펴보겠습니다.